

Instruções

1. Esta atividade pode ser realizada em DUAS ou TRÊS pessoas.
2. A defesa da implementação será realizada no dia indicado no AVA, e a postagem final do código deverá ser realizada até um minuto antes do início da aula. Defesa e/ou entrega em atraso terão desconto progressivo de nota.
3. A implementação poderá ser realizada em qualquer linguagem de programação, sendo que o código deverá ser entregue e apresentado junto com o relatório.
4. Os arquivos da implementação devem ser compactados em um único arquivo ZIP e postados no ambiente AVA 2.0. Lembrem de formar grupo para a postagem do material.
 - a. Alternativamente, pode-se disponibilizar um link acessível para o código-fonte, contendo um arquivo de texto que identifique os integrantes do grupo.
5. A defesa é individual: cada integrante deve ser capaz de explicar qualquer parte do código entregue.
6. Se forem identificados trabalhos com grau de similaridade que caracterize cópia ou adaptação (autorizadas ou não pelos seus autores originais), a nota dos grupos será a nota de um trabalho dividida pelo número de grupos que entregou esses trabalhos similares.

Contexto

Esta é a primeira de duas etapas de um mesmo trabalho. Ao final da segunda etapa, o programa deverá detectar conflitos (hazards) de dados e de controle em um trecho de código executado em um pipeline RISC-V. Para chegar lá, é necessário primeiro extrair da instrução, de forma confiável, quais registradores ela lê e quais ela escreve — e é exatamente isso que esta etapa constrói.

Por esse motivo, a qualidade da decodificação importa mais do que a quantidade de instruções suportadas: um decodificador que informa um rd inexistente em uma instrução de desvio produzirá, na etapa seguinte, um conflito que não existe. Projetem o código pensando que ele será estendido, e não descartado.

Atividade

Elaborar um programa que, a partir de um arquivo de memória de instruções (ROM) em linguagem de máquina, decodifique e caracterize cada instrução nele contida. O programa deve tratar todas as instruções utilizadas nas atividades de programação de RISC-V da disciplina de Arquitetura de Computadores I, incluindo as instruções diretas e as geradas por pseudo-instruções.

R1 – Leitura da entrada

- Ler arquivos com uma instrução por linha, em hexadecimal ou em binário.
- Identificar automaticamente qual dos dois formatos está sendo lido, sem depender de opção de linha de comando.
- Ignorar linhas em branco, linhas de comentário e o prefixo 0x, quando presentes.
- Atribuir a cada instrução o seu endereço (PC), a partir de um endereço-base configurável.

R2 – Classificação e identificação

- Classificar cada instrução em um dos seis formatos: R, I, S, B, U ou J.
- Identificar o mnemônico da instrução, e não apenas o seu formato. Isso exige combinar opcode, funct3 e funct7 — é o que distingue add de sub, srli de srli, lb de lw, e beq de bltu.
- Reportar como inválida qualquer palavra que não corresponda a uma instrução conhecida, informando o endereço em que ela ocorre, sem interromper o processamento do restante do arquivo.

R3 – Extração dos campos

- Separar os campos de cada instrução conforme o seu formato.
- **Informar apenas os campos que existem naquele formato. Uma instrução de tipo B não possui rd, e uma de tipo U não possui rs1, rs2 nem funct3, ainda que os bits correspondentes conttenham algum valor.**

- Reconstruir corretamente os imediatos, com atenção especial aos tipos B e J, cujos bits são armazenados fora de ordem, e aplicar extensão de sinal onde couber.

R4 – Saída

- Apresentar uma lista com o endereço, a palavra original, o formato, o mnemônico e os campos de cada instrução.
- Emitir também a instrução desmontada em assembly, usando os nomes ABI dos registradores (x8 como s0, x2 como sp, e assim por diante).
- Para desvios e saltos, calcular e exibir o endereço absoluto de destino, e não apenas o deslocamento.
- Reconhecer na saída as pseudo-instruções mais comuns, indicando, por exemplo, que addi x0, x0, 0 corresponde a nop e que jalr x0, 0(x1) corresponde a ret.

R5 – Relatório estatístico

- Ao final da listagem, informar a contagem de instruções por formato e o percentual que cada formato representa no programa analisado.
- A partir de uma tabela de CPI por classe de instrução fornecida como entrada, calcular o CPI médio do programa analisado, aplicando a média aritmética ponderada estudada na primeira avaliação.

Formatos de instrução

Para identificar o formato, basta ler os bits 0 a 6 de cada instrução e comparar o opcode com os apresentados na página 49 do guia prático do RISC-V (disponível on-line e na biblioteca da UNIVALI). A tabela a seguir resume quais campos são válidos em cada formato:

Formato	rd	rs1	rs2	funct3	funct7	imediato
Tipo R	sim	sim	sim	sim	sim	—
Tipo I	sim	sim	—	sim	—	12 bits
Tipo S	—	sim	sim	sim	—	12 bits
Tipo B	—	sim	sim	sim	—	13 bits
Tipo U	sim	—	—	—	—	32 bits
Tipo J	sim	—	—	—	—	21 bits

Exemplos de decodificação

Os exemplos abaixo podem ser usados como primeiro teste da implementação:

```

0x00500413  formato = I  mnemônico = addi  rd = 8  rs1 = 0  imm = 5  f3 = 0
              assembly: addi s0, zero, 5

0x00C58633  formato = R  mnemônico = add   rd = 12  rs1 = 11  rs2 = 12
              f3 = 0  f7 = 0
              assembly: add a2, a1, a2

0x0064A423  formato = S  mnemônico = sw    rs1 = 9  rs2 = 6  imm = 8      (sem rd)
              assembly: sw t1, 8(s1)

0xFE628CE3  formato = B  mnemônico = beq   rs1 = 5  rs2 = 6  imm = -8      (sem rd)
              assembly: beq t0, t1, <PC-8>

```

Atenção ao último exemplo: nele, os bits 11 a 7 valem 25. Um decodificador descuidado informaria **rd = 25**, mas o tipo B não possui rd — esses bits são parte do imediato. Erros dessa natureza passam despercebidos nesta etapa e produzem conflitos falsos na etapa seguinte.

Validação

Os códigos de máquina para teste podem ser gerados no simulador RISC-V disponível em <https://riscv-simulator-five.vercel.app/>, por meio do botão "Export code" do editor. Também podem ser utilizados os códigos das atividades de programação de RISC-V da disciplina de Arquitetura de Computadores I.

A implementação será verificada quanto aos seguintes pontos:

- Lê arquivos com dados em hexadecimal e em binário, identificando o formato automaticamente;
- Classifica corretamente as instruções nos seis formatos;
- Identifica o mnemônico de cada instrução;
- Separa os bits de cada instrução corretamente, respeitando quais campos existem em cada formato;
- Reconstrói os imediatos, inclusive os embaralhados dos tipos B e J, com extensão de sinal;
- Apresenta a listagem das instruções com seus dados e a desmontagem em assembly; e
- Apresenta o relatório estatístico com a distribuição por formato e o CPI médio.

Recomenda-se que o grupo monte um conjunto próprio de casos de teste, contendo ao menos uma instrução de cada formato e ao menos um desvio com deslocamento negativo. Esse conjunto deve ser entregue junto com o código.

Distribuição da nota

- Leitura robusta da entrada (R1) — 1,0 ponto
- Classificação nos seis formatos (R2) — 1,5 ponto
- Identificação do mnemônico por opcode, funct3 e funct7 (R2) — 2,0 pontos
- Extração de campos com validade por formato (R3) — 1,5 ponto
- Reconstrução e extensão de sinal dos imediatos (R3) — 2,0 pontos
- Saída legível, com nomes ABI e alvo absoluto dos desvios (R4) — 1,0 ponto
- Relatório estatístico e CPI médio (R5) — 1,0 ponto

A defesa individual poderá ajustar a nota de cada integrante em relação à nota do grupo.

Etapa 2 (apenas para conhecimento)

Na segunda etapa, o programa será estendido para percorrer a sequência de instruções já decodificada e identificar os conflitos de dados e de controle que ocorreriam em um pipeline de cinco estágios, indicando o tipo de conflito, as instruções envolvidas e a quantidade de ciclos de parada necessários. Nenhuma análise de conflitos é exigida nesta primeira etapa.